Homework 10

```
library(RLT)
library(MASS)
set.seed(432)
```

Question 1: Bias-variance trade-off simulation

Random seed: 432. I set it at the top of the document, again before the 100-per-tree study, and again before the MNIST analysis. The test points are generated once and stay fixed, and each of the 100 repetitions draws a fresh training set shared by all tuning settings.

a. Simulation

```
p      <- 5
rho    <- 0.3
Sigma  <- matrix(rho, p, p); diag(Sigma) <- 1
n_tr   <- 200
n_te   <- 200
n_rep  <- 100
ntrees <- 500

f_true <- function(X) X[, 1] + X[, 3] + X[, 5]

Xte  <- mvrnorm(n_te, mu = rep(0, p), Sigma = Sigma)
fte  <- f_true(Xte)

fit_rf <- function(X, y, Xnew, mtry, nmin, n_samp = n_tr) {
  fit <- RLT(X, y, model = "regression",
             ntrees = ntrees, mtry = mtry, nmin = nmin,
             resample.prob = n_samp / nrow(X), resample.replace = TRUE,
```

```
              reinforcement = FALSE, ncores = 1, verbose = 0)
  as.numeric(predict(fit, Xnew)$Prediction)
}

set_nmin  <- data.frame(mtry = 3,     nmin = c(1, 5, 10, 20, 30))
set_mtry  <- data.frame(mtry = 1:5,   nmin = 10)

run_sim <- function(settings, n_samp = n_tr) {

  preds <- lapply(seq_len(nrow(settings)), function(i) matrix(NA_real_, n_rep, n_te))
  for (r in seq_len(n_rep)) {
    X <- mvrnorm(n_tr, rep(0, p), Sigma)
    y <- f_true(X) + rnorm(n_tr)
    for (i in seq_len(nrow(settings))) {
      preds[[i]][r, ] <- fit_rf(X, y, Xte, settings$mtry[i], settings$nmin[i], n_samp)
    }
  }
  out <- settings
  out$bias2 <- sapply(preds, function(P) mean((colMeans(P) - fte)^2))
  out$var   <- sapply(preds, function(P) mean(apply(P, 2, var)))
  out$total <- out$bias2 + out$var
  out
}

res_nmin <- run_sim(set_nmin)
res_mtry <- run_sim(set_mtry)
```

```
knitr::kable(res_nmin, digits = 4, caption = "mtry = 3, varying node size")
```

Table 1: mtry = 3, varying node size

mtry	nmin	bias2	var	total
3	1	0.1033	0.1877	0.2910
3	5	0.1039	0.1828	0.2867
3	10	0.1129	0.1717	0.2846
3	20	0.1486	0.1592	0.3077
3	30	0.2044	0.1552	0.3596

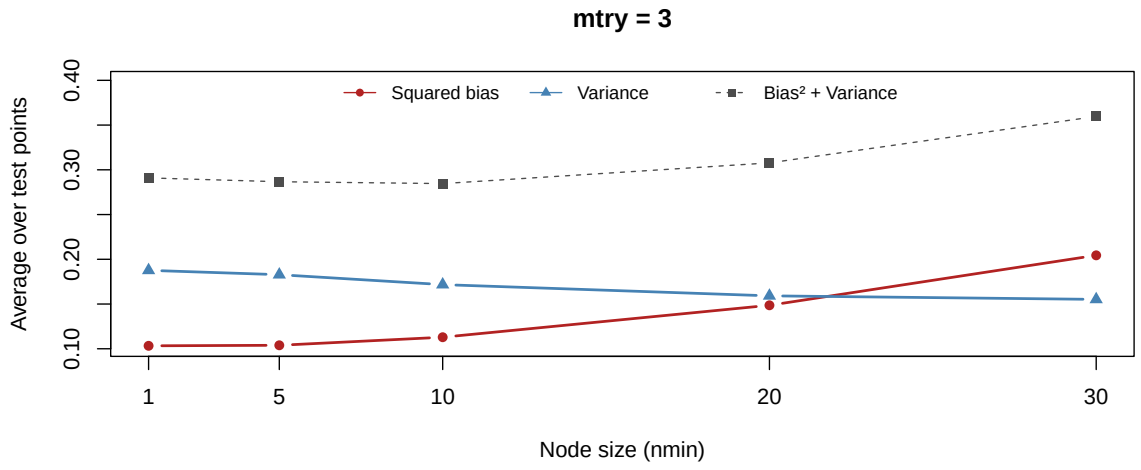
```
knitr::kable(res_mtry, digits = 4, caption = "node size = 10, varying mtry")
```

Table 2: node size = 10, varying mtry

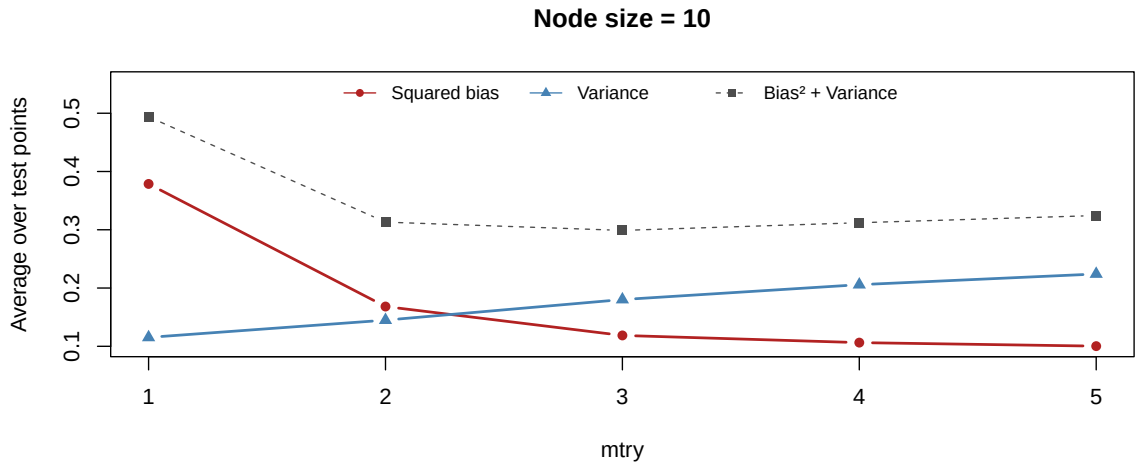
mtry	nmin	bias2	var	total
1	10	0.3787	0.1153	0.4939
2	10	0.1682	0.1449	0.3131
3	10	0.1187	0.1802	0.2989
4	10	0.1063	0.2057	0.3121
5	10	0.1003	0.2241	0.3244

```r
plot_bv <- function(res, xvar, xlab, main) {
  x  <- res[[xvar]]
  yr <- range(c(res$bias2, res$var, res$total))
  yr[2] <- yr[2] + 0.15 * diff(yr)        # room for the legend
  plot(x, res$bias2, type = "b", pch = 16, col = "firebrick", lwd = 2,
       ylim = yr, xlab = xlab, ylab = "Average over test points",
       main = main, xaxt = "n")
  axis(1, at = x)
  lines(x, res$var,   type = "b", pch = 17, col = "steelblue", lwd = 2)
  lines(x, res$total, type = "b", pch = 15, col = "gray30", lty = 2)
  legend("top", c("Squared bias", "Variance", "Bias² + Variance"),
         col = c("firebrick", "steelblue", "gray30"),
         pch = c(16, 17, 15), lty = c(1, 1, 2), bty = "n", horiz = TRUE, cex = 0.8)
}

plot_bv(res_nmin, "nmin", "Node size (nmin)", "mtry = 3")
```



```
plot_bv(res_mtry, "mtry", "mtry", "Node size = 10")
```



b. Discussion

Node size.

Going from node size 1 to 30 roughly doubles the squared bias (0.103 to 0.204), while the variance only drops from 0.188 to 0.155. Averaging 500 trees already removes much of the variance, so there is not much left for large nodes to remove. The bias cost, however, is real: the true function is a smooth linear surface, and a leaf holding around 30 observations averages over a wide range of $x_1 + x_3 + x_5$, so the fit becomes too coarse. The total bias^2 + variance is smallest at node size 10 (0.285), but sizes 1, 5 and 10 are within about 0.006 of each other, which I would call a tie given the simulation noise. Node size 30 is clearly worse (0.360).

`mtry`.

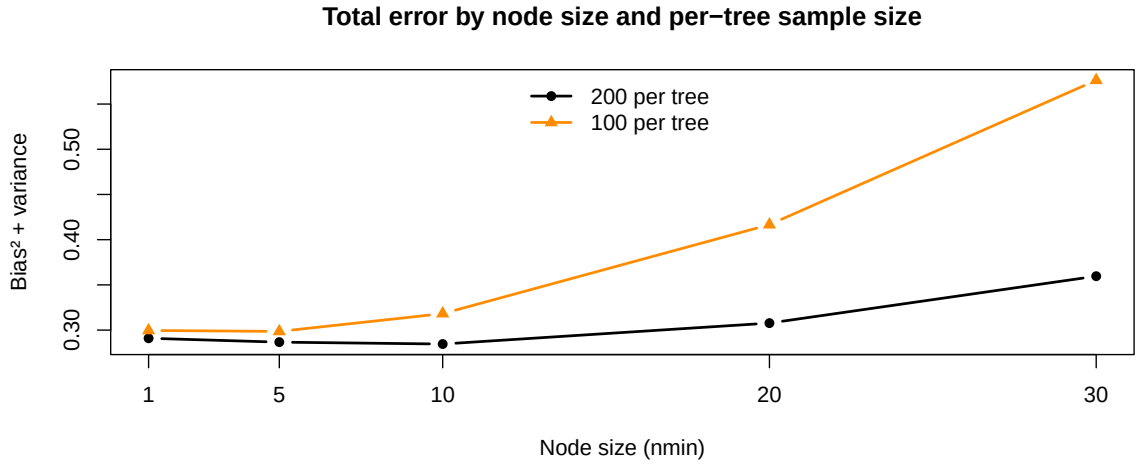
The squared bias falls sharply between `mtry` $= 1$ (0.379) and `mtry` $= 3$ (0.119) and only slightly after that (0.100 at `mtry` $= 5$), while the variance rises steadily from 0.115 to 0.224. With `mtry` $= 1$ the splitting variable is effectively chosen at random, so two of every five splits land on the noise predictors X_2 and X_4 and the trees cannot focus on the signal. A larger `mtry` lets each split find X_1, X_3 and X_5, which reduces bias. The cost is that the trees start to look alike (at `mtry` $= 5$ this is bagging), so averaging them removes less variance, and each fit takes longer. The best total is at `mtry` $= 3$ (0.299), a compromise between the two effects.

```r
set.seed(432)
res_nmin_100 <- run_sim(set_nmin, n_samp = 100)
knitr::kable(res_nmin_100, digits = 4,
             caption = "mtry = 3, varying node size, 100 observations per tree")
```

Table 3: mtry = 3, varying node size, 100 observations per tree

mtry	nmin	bias2	var	total
3	1	0.1627	0.1369	0.2996
3	5	0.1674	0.1312	0.2985
3	10	0.1940	0.1242	0.3182
3	20	0.2969	0.1197	0.4166
3	30	0.4521	0.1242	0.5763

```r
yr <- range(c(res_nmin$total, res_nmin_100$total))
plot(res_nmin$nmin, res_nmin$total, type = "b", pch = 16, lwd = 2, ylim = yr,
     xlab = "Node size (nmin)", ylab = "Bias² + variance", xaxt = "n",
     main = "Total error by node size and per-tree sample size")
axis(1, at = res_nmin$nmin)
lines(res_nmin_100$nmin, res_nmin_100$total, type = "b", pch = 17, lwd = 2, col = "darkorange")
legend("top", c("200 per tree", "100 per tree"), col = c("black", "darkorange"),
       pch = c(16, 17), lwd = 2, bty = "n")
```



```r
cat("Best nmin with 200 per tree:", res_nmin$nmin[which.min(res_nmin$total)], "\n")
```

```
Best nmin with 200 per tree: 10
```

```r
cat("Best nmin with 100 per tree:", res_nmin_100$nmin[which.min(res_nmin_100$total)], "\n")
```

```
Best nmin with 100 per tree: 5
```

Sampling 100 observations per tree.

With 100 draws per tree the best node size is 5 instead of 10, so it moved toward smaller nodes, as I expected. The reasoning is that each tree sees less data, so a node of a given size is a larger share of the tree's sample and the tree is already coarser. The variance is also already lower (about 0.12–0.14 versus 0.16–0.19), so there is less to gain from large nodes. The clearest evidence in the table is not the location of the minimum but how much faster the bias grows: at node size 30 the squared bias is 0.452 with 100 draws per tree, against 0.204 with 200. I would not lean on the 5-versus-10 comparison itself, because with 200 draws per tree the totals for node sizes 1, 5 and 10 were nearly tied, and 100 repetitions cannot separate such small gaps.

Question 2: Tuning random forests for MNIST

Data preparation

```r
set.seed(432)
mnist <- read.csv(gzfile("data/mnist-first2000.csv.gz"))
keep  <- c(1, 6, 7)

train_raw <- mnist[1:1000, ]
test_raw  <- mnist[1001:2000, ]
train_raw <- train_raw[train_raw$Digit %in% keep, ]
test_raw  <- test_raw[test_raw$Digit %in% keep, ]

ytr <- factor(train_raw$Digit, levels = keep)
yte <- factor(test_raw$Digit,  levels = keep)

Xtr_pix <- as.matrix(train_raw[, -1])
Xte_pix <- as.matrix(test_raw[, -1])
```

```r
pca <- prcomp(Xtr_pix, center = TRUE, scale. = FALSE, rank. = 20)
Xtr <- pca$x[, 1:20]
Xte2 <- scale(Xte_pix, center = pca$center, scale = FALSE) %*% pca$rotation[, 1:20]
colnames(Xte2) <- colnames(Xtr)

c(n_train = nrow(Xtr), n_test = nrow(Xte2))
```

```
n_train  n_test
    327     317
```

```r
table(ytr)
```

```
ytr
  1   6   7
116  94 117
```

```r
table(yte)
```

```
yte
  1   6   7
104 106 107
```

```r
ntrees2 <- 1000
n2      <- nrow(Xtr)

fit_cls <- function(mtry = NULL, nmin = NULL, importance = FALSE) {
  if (is.null(mtry)) mtry <- max(1, as.integer(ncol(Xtr) / 3))
  if (is.null(nmin)) nmin <- max(1, as.integer(log(nrow(Xtr))))
  RLT(x = Xtr, y = ytr, model = "classification",
      ntrees = ntrees2,
      mtry = as.numeric(mtry), nmin = as.numeric(nmin),
      resample.prob = 1, resample.replace = TRUE,
      reinforcement = FALSE, importance = importance,
      ncores = 1, verbose = 0)
}

oob_error <- function(fit, y = ytr) {
  pr <- as.character(fit$Prediction)
  ok <- !is.na(pr)
```

```
  err <- mean(pr[ok] != as.character(y[ok]))
  if (is.numeric(fit$Error) && length(fit$Error) == 1 &&
      abs(err - fit$Error) > 1e-6)
    warning("Computed OOB error (", round(err, 4),
            ") differs from fit$Error (", round(fit$Error, 4), ")")
  err
}
```

a. Default forest

```
set.seed(432)
fit_default <- fit_cls()
err_default <- oob_error(fit_default)
err_default
```

```
[1] 0.03669725
```

With RLT's default settings (`mtry` $= \max(1, \lfloor p/3 \rfloor) = 6$ and `nmin` $= \lfloor \log n \rfloor$), the **OOB misclassification rate is 0.0367** (3.67%).

b. Tuning

```
set.seed(432)
grid <- expand.grid(mtry = c(1, 2, 4, 6, 8, 12, 20),
                    nmin = c(1, 3, 5, 10, 20, 40))
grid$oob_error <- NA_real_
for (i in seq_len(nrow(grid))) {
  fit <- fit_cls(mtry = grid$mtry[i], nmin = grid$nmin[i])
  grid$oob_error[i] <- oob_error(fit)
}

tab <- reshape(grid, idvar = "mtry", timevar = "nmin", direction = "wide")
names(tab) <- sub("oob_error.", "nmin = ", names(tab))
knitr::kable(tab, digits = 4, caption = "OOB misclassification rate by (mtry, node size)")
```

Table 4: OOB misclassification rate by (mtry, node size)

mtry	nmin = 1	nmin = 3	nmin = 5	nmin = 10	nmin = 20	nmin = 40
1	0.0306	0.0183	0.0214	0.0275	0.0214	0.0398
2	0.0306	0.0367	0.0367	0.0428	0.0398	0.0398
4	0.0367	0.0367	0.0398	0.0367	0.0398	0.0398
6	0.0367	0.0367	0.0367	0.0367	0.0367	0.0367
8	0.0398	0.0398	0.0398	0.0398	0.0398	0.0398
12	0.0459	0.0428	0.0489	0.0459	0.0459	0.0459
20	0.0489	0.0489	0.0520	0.0489	0.0520	0.0550

```
best <- grid[which.min(grid$oob_error), ]
ties <- grid[grid$oob_error == best$oob_error, ]
ties
```

```
  mtry nmin  oob_error
8    1    3 0.01834862
```

```
best <- ties[order(-ties$nmin, ties$mtry), ][1, ]
best
```

```
  mtry nmin  oob_error
8    1    3 0.01834862
```

The selected setting (smallest OOB error, ties broken toward the larger node size and then the smaller `mtry`) is `mtry` = 1 with node size 3, with OOB error 0.0183 (6 of 327 observations).

```
set.seed(432)
fit_best <- fit_cls(mtry = best$mtry, nmin = best$nmin, importance = TRUE)
oob_error(fit_best)
```

```
[1] 0.03058104
```

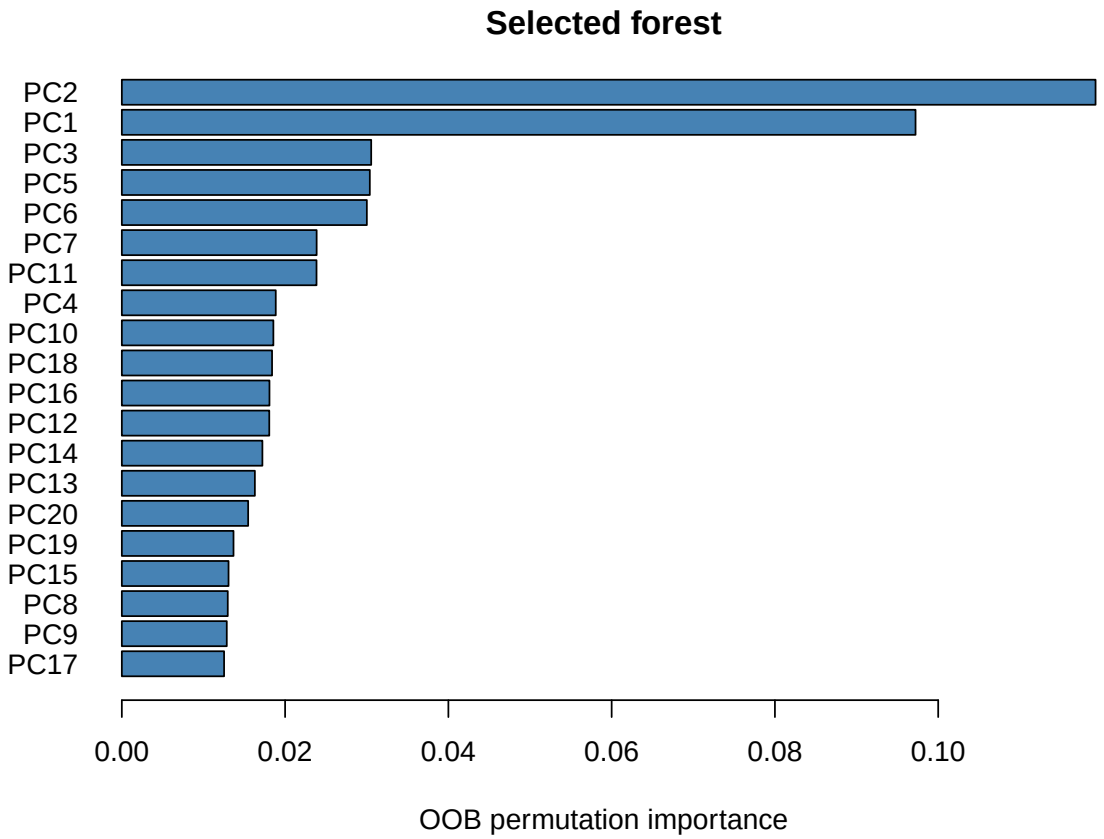
```
vi <- fit_best$VarImp
vi <- if (is.matrix(vi) || is.data.frame(vi)) as.numeric(vi[, 1]) else as.numeric(vi)
names(vi) <- colnames(Xtr)
vi_sorted <- sort(vi, decreasing = TRUE)

knitr::kable(data.frame(PC = names(vi_sorted), Importance = vi_sorted, row.names = NULL),
             digits = 4, caption = "OOB permutation variable importance")
```

Table 5: OOB permutation variable importance

PC	Importance
PC2	0.1193
PC1	0.0972
PC3	0.0305
PC5	0.0304
PC6	0.0300
PC7	0.0239
PC11	0.0238
PC4	0.0189
PC10	0.0186
PC18	0.0184
PC16	0.0181
PC12	0.0181
PC14	0.0172
PC13	0.0163
PC20	0.0155
PC19	0.0137
PC15	0.0131
PC8	0.0130
PC9	0.0129
PC17	0.0125

```
par(mar = c(4, 5, 2, 1))
barplot(rev(vi_sorted), horiz = TRUE, las = 1, col = "steelblue",
        xlab = "OOB permutation importance", main = "Selected forest")
```



The importance plot is dominated by PC2 (0.119) and PC1 (0.097), well ahead of everything else. The next components are much smaller, and the order does not simply follow the variance order: PC5, PC6, PC7 and even PC11 rank above PC4. Refitting the selected setting to compute variable importance gives an OOB error of 0.0306 (10 observations), against 0.0183 (6 observations) in the grid. The gap comes from the forest's own randomness, and also from the fact that I took the smallest of 42 noisy estimates, so 0.0183 is probably optimistic. I would treat the selected setting as one of several about equally good ones, not as a sharp winner.

- The tuning surface is fairly flat. Most settings with `mtry` ≤ 8 sit between roughly 2% and 4.3% error, and most neighboring cells differ by only one to three observations out of 327 (one observation is about 0.3 percentage points). The one clear trend is that the error rises with `mtry`: it is about 4.3–4.9% at `mtry` $= 12$ and 4.9–5.5% at `mtry` $= 20$, against 1.8–4.0% at `mtry` $= 1$. Node size matters much less for `mtry` ≥ 4 (the rows for `mtry` $= 6$ and 8 are constant across node sizes). Its biggest effect is at `mtry` $= 1$, where node size 40 is clearly worse than the smaller sizes. The default setting (`mtry` $= 6$, 0.0367) is only a few observations behind the best cell. A plausible reason for the trend is that the principal components are uncorrelated and most of the signal sits in the first few, so with a large `mtry` most splits can use PC1 or PC2 and the trees become very

similar, while a small `mtry` forces more variety between trees. I only have the pattern, not a test of this explanation.

- The OOB error is not perfectly honest, since the PCA was fit on all training pixels, including the ones that are out of bag for a given tree. I think the effect is small, and part (c) gives a clean check.

c. Test performance

```
pred <- predict(fit_best, Xte2)$Prediction
if (is.matrix(pred)) pred <- levels(ytr)[max.col(pred)]
pred <- factor(as.character(pred), levels = levels(ytr))

cm <- table(Predicted = pred, Actual = yte)
cm
```

```
         Actual
Predicted   1   6   7
        1 102   0   0
        6   0 106   3
        7   2   0 104
```

```
test_err <- mean(pred != yte)
test_err
```

```
[1] 0.01577287
```

The test misclassification rate is 0.0158 (5 errors out of 317 test observations). Three 7s were predicted as 6, and two 1s were predicted as 7. The test error is in the same range as the OOB error from tuning (0.0183 in the grid, 0.0306 for the refit). With only a handful of errors on either side, I would not claim that one is really lower than the other. The takeaway is that tuning on the OOB error did not seem to overfit, since the test labels were not used for any choice.

Question 3: Planning a random-forest analysis skill

a. Approach

I would keep the skill short. A capable AI already knows how to fit a forest, use OOB error, tune `mtry` and node size, and report accuracy, so spelling all of that out mostly adds length,

and a long recipe can even lock the AI into a worse approach than it would have picked itself. I would spend the words on the things the AI cannot guess: my preferred package and parameter names, how I want the data split, which tables and figures I expect, how I want results interpreted, and how long and how formal the report should be. I would also add the few mistakes I have seen before, such as tuning on the test set or reporting only the winning setting. If the AI were weaker, I would add a short checklist with a sentence of reasoning for each step. Whichever version I use, I would test the skill on a small dataset and delete any line that does not change the output. I would also tell the AI that it can deviate from the skill when the data call for it, as long as it explains why.

b. Ten instructions, from basic to personal

1. Look at the data first: sizes, variable types, missing values, and the class balance or the response distribution. State whether the task is regression or classification.
2. Keep the test data out of every decision. Tune with OOB error or cross-validation, and use the test set only once at the end.
3. Use a named package, set a seed, and report the seed, the package version, and the number of trees (at least 500).
4. Tune at least `mtry` and node size over a grid that you write down, and show the whole grid, not just the winner.
5. Report the right metrics: misclassification rate and a confusion matrix for classification, RMSE and R^2 for regression, and compare with a simple baseline.
6. Include permutation-based variable importance, and mention that it can be unreliable when predictors are strongly correlated.
7. Give each main result (tuning grid, importance, final performance) its own table or figure with clear labels.
8. Organize the report as data, method, tuning, results, conclusion, with the code included, so that it renders from a single file.
9. Write the interpretation in plain language. Say what was learned about the data, and say when differences between settings are small enough to be noise.
10. Personal style: about two pages, the final test result and the chosen tuning values in the first paragraph, simple base-R plots in one consistent color scheme, and a short closing list of limitations and next steps.